

# LLVM IR: Estructuras de control

## Generando LLVM IR para estructuras de control

A manera de ejemplo, les propongo que analicemos el código LLVM IR generado a partir de la implementación en Lenguaje C del cálculo del factorial, como se muestra a continuación.

```
int factorial(int n) {  
    int f = 1, i = 1;  
    while (i <= n) {  
        f *= i;  
        i += 1;  
    }  
    return f;  
}
```

Decidí usar esta versión no recursiva, para poder ilustrar como se puede construir cada uno de los bloques involucrados, particularmente, aquellos asociados con la estructura de control "while". A continuación, incluyo el código completo en python (LLVMLite) que genera el código intermedio correspondiente al programa en Lenguaje C.

```
from llvmlite import ir  
  
intType = ir.IntType(32)  
floatType = ir.FloatType()  
module = ir.Module(name="prog")  
  
# int factorial(int n) {  
fnty = ir.FunctionType(intType, [intType])  
func = ir.Function(module, fnty, name='factorial')  
  
n_arg = func.args[0]  
n_arg.name = 'n'  
  
entry = func.append_basic_block('entry')  
builder = ir.IRBuilder(entry)  
  
# int f = 1, i = 1;  
one_constant = ir.Constant(intType, 1)  
f_var = builder.alloca(intType, name="f")  
i_var = builder.alloca(intType, name="i")  
builder.store(one_constant, f_var)  
builder.store(one_constant, i_var)  
  
whileHead = func.append_basic_block('while-head')  
whileBody = func.append_basic_block('while-body')  
whileExit = func.append_basic_block('while-exit')
```

```

builder.branch(whileHead)

# while (i <= n)
builder.position_at_start(whileHead)
builder.cbranch(
    builder.icmp_signed('<=', builder.load(i_var, 'i'), n_arg),
    whileBody, whileExit
)

# { f *= i; i += 1; }
builder.position_at_start(whileBody)
builder.store(
    builder.mul(builder.load(f_var, 'f'), builder.load(i_var, 'i')),
    f_var
)
builder.store(
    builder.add(builder.load(i_var, 'i'), one_constant),
    i_var
)
builder.branch(whileHead)

# return f
builder.position_at_start(whileExit)
tmp = builder.load(f_var)
builder.ret(tmp)

print(module)

```

Para poder ejecutar el código antes generado, modifiqué el script python usado para los ejemplos previos. Note que podemos variar el valor del parámetro "n" y así verificar que el código se comporta como esperado.

```

import runtime as rt
from ctypes import CFUNCTYPE, c_int

engine = rt.create_execution_engine()
mod = rt.compile_ir(engine, str(module))
func_ptr = engine.get_function_address("factorial")

cfunc = CFUNCTYPE(c_int, c_int)(func_ptr)
n = 5
res = cfunc(n)
print(f"factorial({n}) =", res)

```

## Variables locales

```

one_constant = ir.Constant(intType, 1)
f_var = builder.alloca(intType, name="f")
builder.store(one_constant, f_var)

```

**Declaración de una variable.** La declaración de una variable se realiza con `builder.alloca()`.

Note que se debe especificar el tipo de dato de la variable y, opcionalmente, dar un nombre a la variable. Note que es una práctica común el dar un valor inicial a todas las variables, usando

`builder.store()`.

```
builder.load(i_var, 'i')
```

**Acceso al valor de una variable.** Se realiza con `builder.load()`. Note que el segundo parámetro en la llamada es opcional. Yo recomiendo incluir el nombre de la variable pues esto facilita la comprensión del código generado.

```
builder.store(one_constant, f_var)
```

**Actualización del valor de una variable.** Se realiza con `builder.store()`.

## Parámetros

```
fnty = ir.FunctionType(intType, [intType])
func = ir.Function(module, fnty, name='factorial')

n_arg = func.args[0]
n_arg.name = 'n'
```

**Declaración de un parámetro.** La instanciación de la firma de la función puede incluir el tipo de dato de cada uno de los parámetros en la lista del segundo argumento de `FunctionType`. Posteriormente, podremos acceder un manejador de cada uno de los parámetros usando `func.args[i]`, donde `i` corresponde a la posición del parámetro. Con cada manejador podemos, por ejemplo, dar un nombre al parámetro correspondiente.

```
builder.cbranch(
    builder.icmp_signed('<=', builder.load(i_var, 'i'), n_arg),
    whileBody, whileExit
)
```

**Acceso al valor de un parámetro.** Note que el valor del parámetro se considera inmutable. Así, en el ejemplo, accedemos al valor de `n_arg` directamente. En muchos lenguajes de programación, sin embargo, es posible modificar el valor de los parámetros. Para lograr esto en LLVM IR, podemos crear una variable local y copiar el valor del parámetro a esta variable, al principio de la función subyacente.

## Bloques de código

```
whileHead = func.append_basic_block('while-head')
```

**Creación de un bloque.** Dependiendo de la estructura de control, se deberán crear uno o más bloques de código. En el ejemplo usado en la sesión, se crearon 3 bloques, mismos que son necesarios para una estructura de control "while".

Note que podemos usar el mismo nombre de bloque en el mismo programa. LLVM Lite generará un nombre temporal con un sufijo para distinguir cada uno de los bloques, en caso necesario.

```
builder.branch(whileHead)
```

**Salto incondicional a un bloque.** En el ejemplo, LLVM Lite generará el código para realizar un salto incondicional para entrar al bloque de código especificado.

```
builder.cbranch(  
    builder.icmp_signed('<=', builder.load(i_v  
ar, 'i'), n_arg),  
    whileBody, whileExit  
)
```

**Salto condicional.** Para algunas estructuras de control, deberemos generar código para realizar saltos condicionales, esto es, saltos a bloques que dependen del valor booleano regresado por una expresión. Este es el caso de un IF/ELSE o del WHILE. En el while, por ejemplo, debemos entrar al cuerpo del ciclo si la condición subyacente es verdadera y salir del ciclo en caso contrario. Note que `builder.cbranch()` en el ejemplo, recibe el resultado de una comparación (`i < n`) y la referencia a los dos bloques previamente instanciados.

```
builder.position_at_start(whileBody)
```

**Activación de un bloque para generación de código.** Previo a emitir el código de un bloque, debemos activarlo usando `builder.position_at_start()`.